

CS179G Backend Testing Guide

A beginner-friendly walkthrough for testing the MySQL-to-Express integration and connecting the React frontend to the N-gram chart data.

FRONTEND	BACKEND	DATABASE
React localhost:3000	Express API localhost:5000	MySQL ngram_db

Current status

The API connection has been verified. If an endpoint returns an empty data array, the integration is working but the related Part 2 table does not contain processed rows yet.

Quick test

1 Start MySQL

Use MySQL Workbench or the Windows Services application and confirm the MySQL server is running.

2 Start the backend

Open PowerShell in the project's backend folder and run the commands below. Keep the terminal open.

```
npm install
npm start
```

3 Open the health endpoint

Visit `http://localhost:5000/api/health` in a browser.

```
{ "database": "connected" }
```

4 Test chart data

Open the trend and decade endpoints shown on the next page. A non-error JSON response confirms that React can call the API.

1. Confirm the MySQL tables

Open PowerShell and connect to MySQL. The password is typed at the prompt and should never be placed in frontend code.

```
mysql -u root -p
```

Run these SQL statements:

```
USE ngram_db;
SHOW TABLES;
SELECT COUNT(*) FROM word_year_stats;
SELECT COUNT(*) FROM decade_top_words;
exit;
```

2. Test every API route

Purpose	Address	Chart use
Database health	/api/health	Connection status
Keyword trend	/api/trends?keyword=database	Line chart
Filtered trend	/api/trends?keyword=database&startYear=1900&endYear=2009	Line chart
Available decades	/api/decades	Dropdown
Top words	/api/top-words?decade=1990&limit=10	Bar chart

Expected trend response

```
{
  "keyword": "database",
  "data": [
    { "year": 1980, "match_count": 55 },
    { "year": 1981, "match_count": 8255 }
  ]
}
```

Expected top-words response

```
{
  "decade": 1990,
  "data": [
    { "word": "the", "total_matches": 500000, "rank": 1 }
  ]
}
```

3. Optional end-to-end test data

Use this only when the Part 2 tables are empty. These records use a distinctive test word and a decade outside the Google Books dataset, making them easy to remove afterward.

Before inserting

Connect to MySQL, select `ngram_db`, and make sure you are working in the intended local database.

Insert temporary rows

```
INSERT INTO word_year_stats
  (word, year, match_count, page_count, volume_count)
VALUES
  ('backendtestword', 2000, 10, 5, 2),
  ('backendtestword', 2001, 25, 12, 4),
  ('backendtestword', 2002, 40, 20, 8);

INSERT INTO decade_top_words
  (decade, word, total_matches, word_rank)
VALUES
  (2090, 'backendtestword', 1000, 1),
  (2090, 'sampleword', 800, 2);
```

Test the inserted rows

```
http://localhost:5000/api/trends?keyword=backendtestword
http://localhost:5000/api/top-words?decade=2090
```

Remove the temporary rows

```
DELETE FROM word_year_stats
WHERE word = 'backendtestword';

DELETE FROM decade_top_words
WHERE decade = 2090;
```

Real data dependencies

The trend route reads `word_year_stats`, which is produced by `src/process_ngrams.py`. The decade route reads `decade_top_words`, which is produced by `src/process_decade_top.py` or `src/process_ngrams.py`.

4. Frontend handoff

The frontend must call Express. It must never connect directly to MySQL or contain a database password.

Run both applications

Terminal 1 - backend	Terminal 2 - frontend
cd backend npm start	cd frontend npm start
http://localhost:5000	http://localhost:3000

React keyword request

```
async function getWordTrend(keyword) {
  const url = 'http://localhost:5000/api/trends?keyword=' +
    encodeURIComponent(keyword);
  const response = await fetch(url);

  if (!response.ok) {
    throw new Error('Could not load the word trend');
  }

  const result = await response.json();
  return result.data;
}
```

Line-chart mapping

```
const labels = data.map(item => item.year);
const values = data.map(item => item.match_count);
```

Popular-words request

```
async function getTopWords(decade) {
  const response = await fetch(
    `http://localhost:5000/api/top-words?decade=${decade}&limit=10`
  );
  if (!response.ok) throw new Error('Could not load popular words');
  const result = await response.json();
  return result.data;
}
```

Bar-chart mapping

```
const labels = data.map(item => item.word);
const values = data.map(item => item.total_matches);
```

5. Frontend rules and troubleshooting

Frontend rules

- Use `encodeURIComponent ( keyword )` before adding user text to a URL.
- Check `response.ok` before reading chart data.
- Treat an empty data array as a valid result and show a clear 'No data found' message.
- For a line chart, use `year` as labels and `match_count` as values.
- For a bar chart, use `word` as labels and `total_matches` as values.
- Never copy `backend/.env` into React or commit it to Git.

Common problems

Problem	What to check
Database says disconnected	Start MySQL and check the values in <code>backend/.env</code> .
The browser cannot reach port 5000	Keep <code>npm start</code> running in the backend terminal.
The API returns data: <code>[]</code>	The query worked, but the Part 2 table has no matching rows.
React reports a CORS error	Use React on port 3000 or update <code>FRONTEND_URL</code> in <code>backend/.env</code> .
A teammate cannot use localhost	<code>localhost</code> refers to that teammate's computer, not yours.

Working on separate computers

The current project uses local databases. Each teammate must run MySQL and the backend locally, or the group must later deploy one shared backend. A React app on another computer cannot reach your `localhost` address.

Final verification checklist

- ☐ MySQL is running.
- ☐ The backend health route says connected.
- ☐ The required Part 2 tables contain rows.
- ☐ Trend and top-word endpoints return JSON.
- ☐ React uses port 3000 and the backend uses port 5000.
- ☐ No database password appears in frontend code or Git.